

ER Diagram Design Explanation

1. Overview

The purpose of our ER Diagram is to model a database for a hotel that specializes in single customers that are looking for a place to stay during their active vacation that usually involves skiing. The key functionalities that the database should manage include managing reservations, tracking packages, ski pass purchases, and having data available to query for analytics.

2. Entity Choices

The **eight entities** we chose to diagram are listed below with a short description:

- **Customer:** represents individuals making reservations.
- **Reservation:** tracks booking periods, package selected, total costs, and the status of the reservation.
- **Package:** bundles room and ski pass options at a fixed price.
- **Room:** holds specific room details (type, floor, number).
- **SkiPass:** represents passes that grant access to ski resorts.
- **SkiResort:** stores info about each resort (difficulty, location, size).
- **Hotel:** Hotel that services the customers. This entity can store analytics and other data in the future.
- **Transportation:** Transportation offered by the hotel to the various ski resorts, with them mainly being offered as shuttle buses or exclusive Taxi's.

Each of these entities have multiple attributes and real-world relationships to each other, which is why we decided to model them as entities. The hotel entity was included even though we only want to model a database for a single hotel to store the hotel's name, address, and for storing data that could be related to the hotel in the future, such as staff or other information. The other entity to comment on is the transportation entity, which represents shuttle buses or taxis offered by the hotel to the various ski resorts around it.

Some entities that we considered adding, but thought was not necessary for the use case requested for the database that we modeled were an entity for staff, Ski Lifts, Ski Slopes, and a person generalization entity that would have customers, staff, and maybe administrators and sub entities.

3. Relationships

The **eight relationships** that connect our entities are listed below:

- **Customer ↔ Reservation (1:*)** => Each customer can make one to many reservations, while each reservation must have a customer related to it **(1:1)**.
- **Reservation ↔ Package (0:1)** => Each reservation can have zero or one package related to it, while each package can have zero to many reservations associated with it **(0:*)**.
- **Reservation ↔ Room (1:1)** => Each reservation has a room, while each room can have zero to many reservations **(0:*)**. This includes past reservations made to this room.
- **Package ↔ Room (1:1)** => Each package has a room, while each room can have zero to many packages **(0:*)**.
- **Hotel ↔ Reservation (0:*)** => The hotel has zero to many reservations, and each of these reservations belong to a single hotel **(1:1)**.
- **Package ↔ Ski Pass (1:1)** => Each package includes a single ski pass, and each ski pass is included in zero to many packages **(0:*)**.
- **Package ↔ Transportation (0:*)** => Each package offers zero to many transportations and each transportation is offered by zero to many packages **(0:*)**.
- **Ski Pass ↔ Ski Resort (1:*)** => Each ski pass has 1 to many ski resorts and each ski resort has one to many ski passes **(1:*)**.
 - In real life a ski pass can sometimes allow access to multiple ski resorts that are in the same area.

4. Attributes Design

Each entity has comments on its attributes and design choices below:

- **Hotel:** Has attributes **name** and **address**. It does not have a primary key because the database is designed for a single hotel that already is unique.
- **Reservation:** Its primary key is **reservationId**. The **totalCost** attribute is a derived attribute that is calculated from the other entities that have costs associated with them. The **status** attribute can keep track of the status of the reservation and whether it has been cancelled. The last two attributes

checkInData and **checkOutDate** are for the date and time when the customer can check in and must check out.

- **Customer:** Its primary key is **customerId**. The **paymentInfo** attribute is a composite attribute that is composed of multiple attributes regarding the payment information of the customer, namely **billingAddress**, **cardHolderName**, **expirationDate**, **cardIssuer** and **cardNumber**. The **email** attribute is multi-valued because a customer can have a regular email address and a work email address. The **name**, **address**, **phoneNumber** and **age** attributes hold the customer's personal information that could be valuable for trend analyses.
- **Package:** Its primary key is **packageId**. The **name** attribute is used for advertising the package (e.g. "Alpine Luxe"). The **description** attribute contains a general overview of the package as a whole. The **price** attribute is the cost of the package itself.
- **Room:** Its primary key is **roomNumber** and it has a **price**, **type**, and **availability** attribute. The first digit of the **roomNumber** primary key includes the floor that the room is on in the hotel, which is where the **floor** attribute is derived from. The **availability** attribute determines whether the room is currently available to be reserved or is currently occupied or under maintenance.
- **Ski Pass:** Its primary key is **skiPassId** and it has the **price** and **validityPeriod** attributes. The **validityPeriod** is a composite attribute made up of the **startDate** and **endDate** attributes.
- **Ski Resort:** Its primary key is **resortId** and it has a **location** and **name** attribute. It also has a **difficultyLevel** attribute and **size** attribute, with the **size** attribute being a composite attribute made up of **skiLiftsCount** and **slopesCount**.
- **Transportation:** Its primary key is **transportId** and it has a **price** and **type** of transportation attribute. Furthermore, it has a **timetable** attribute that we considered making its own entity, but for simplicity decided to leave it as is.

5. Assumptions Made

Some of the assumptions that we have made are that a **customer** must have at least one **reservation** and they can't have zero **reservations**. Another one was for the **customer** to be able to reserve a room without buying a **package**. Further assumptions

include packages not being customizable and there not being the need or option to add multiple ski passes to a single package, which would only make sense if it is a very long ski vacation. It can also be considered whether a reservation can include more than a single package or room in the event a customer wants to switch between or upgrade their rooms during their stay.

6. Possible Extensions

Some possible extensions include:

- Having an entity for different categories of rooms, which could be related to the **room** entity. This would enable the functionality of changing the price for all rooms that are in the same category. The option to dynamically change the price of individual rooms could also be left open by leaving the **price** attribute attached to the **room** entity and using it when we want to change the price of the room to something other than what the normal price for that category of room is. A real-world example would be to make rooms that are on the top floor more expensive.
- Having entities for both the **availability** and **timetable** attributes. This would allow the hotel to reserve rooms and packages in advance, but it would require an entity that has data on all the current and future reservations for a certain room. The **timetable** attribute would similarly have a schedule for each mode of transportation.
- Adding the **person** generalization entity to be able to have **customers** and potentially an entity such as **staff** as sub entities. Furthermore, this would work well with each transportation having an **operator** entity that is also a sub entity of **staff**.